

■ JARVIS OS

MVP in 45 Days

Build a Windows AI Agent that opens apps, controls the browser, sends WhatsApp messages, reads documents, and understands voice commands.

4 Phases	45 Days	15+ Modules	Resume-Ready by Day 30
----------	---------	-------------	---------------------------

Based on your 365-day roadmap — compressed into a 45-day MVP sprint you can add to your resume immediately, then expand over the next 12 months.

Strategy: Build the core loop first → add intelligence → polish for demo → keep adding features.

Table of Contents

Phase 1 · ■ Setup & Core Automation Engine · Days 1–10

Week 1 — Toolchain + First Automations (Days 1–7)

Week 1.5 — Browser Control Basics (Days 8–10)

Phase 2 · ■ LLM Brain — Natural Language Understanding · Days 11–20

Week 2 — LLM Integration + Tool Calling (Days 11–17)

Week 2.5 — WhatsApp & Messaging (Days 18–20)

Phase 3 · ■■ System Control & Advanced Automation · Days 21–30

Week 3 — System + App Deep Control (Days 21–27)

Week 3.5 — UI + Cleanup (Days 28–30)

Phase 4 · ■ Production Polish & Advanced Features · Days 31–45

Week 4 — Memory + Safety + Advanced Browser (Days 31–37)

Week 5 — Packaging, Distribution & Portfolio (Days 38–45)

Tech Stack Overview

Resume Bullet Points

After Day 45 — What to Build Next (Phase 5–8)

How to Use This Roadmap

This is a sprint — not a study plan. Every day you SHIP something that works.

■ Tasks	What to install and learn (1–2 hours)
■■ Build	What to code and ship that day (2–4 hours)
■ Outcome	What must be working by end of day
■ Tip	A specific technical tip to avoid common mistakes

Daily schedule: 3–5 hours/day. Morning: learn the concept. Afternoon/Evening: code it. Night: push to GitHub.

If you get stuck: search the error message, check the library docs, ask an AI for the specific error — then move on. Don't get blocked for more than 20 minutes on one thing.

Phase 1 ■ Setup & Core Automation Engine

Days 1–10

Goal: Install every tool, build your core Python automation layer, and get files + apps opening on command.

Week 1 | Toolchain + First Automations | Days 1–7

Day 1 | Dev Environment Setup

- Install Python 3.12, Node.js 20, Git, VS Code
- Create project: `jarvis-agent/` with `venv`
- Install core libs: `pyautogui`, `pywinauto`, `pynput`, `pytsx3`, `SpeechRecognition`
- Install: `requests`, `python-dotenv`, `loguru`, `rich`

■■ **Build** Run `hello_jarvis.py` — print a message and open Notepad programmatically.

■ **Outcome** Working dev environment, first automated window open

■ **Tip** *Keep a `DEVLOG.md` from Day 1 — note what you built each day. You'll use this for your resume.*

Day 2 | File System Automation

- Master `pathlib` — list, create, move, delete, watch files
- Build `file_agent.py`: `open_file()`, `list_dir()`, `search_files()`, `organize_downloads()`
- Add watchdog file watcher — detect new files in `~/Downloads`
- Test: copy a file, rename it, move it, get its metadata

■■ **Build** Script that watches `~/Downloads`, detects new file, auto-moves it to the correct folder (PDFs → `~/Documents/PDFs`, Images → `~/Pictures`, etc.)

■ **Outcome** Full file system control via Python

Day 3 | App Launcher & Window Control

- Learn `subprocess.Popen` to open any `.exe`
- Learn `pywinauto` — `find_window()`, `set_focus()`, `move_window()`, `resize()`
- Build `app_agent.py`: `open_app(name)`, `close_app(name)`, `focus_app(name)`, `list_open_apps()`
- Build an app registry dict: `{'chrome': 'C:/...chrome.exe', 'notepad': 'notepad.exe', ...}`

■■ **Build** `open_app('whatsapp')` → opens WhatsApp Desktop. `open_app('chrome')` → opens Chrome. Works for any registered app.

■ **Outcome** Any app opens/closes/focuses on command

■ **Tip** *Add 20 common apps to your registry today — WhatsApp, Chrome, VS Code, Notepad, Explorer, Calculator, Spotify, Discord, Telegram.*

Day 4

Keyboard & Mouse Automation

- Learn pyautogui: `typewrite()`, `hotkey()`, `moveTo()`, `click()`, `screenshot()`
- Learn `pynput` for global hotkeys
- Build `input_agent.py`: `type_text()`, `press_key()`, `click_at()`, `scroll()`, `hotkey()`
- Test: open Notepad, type 'Hello World', select all, copy, paste in another window

■ ■ **Build** Macro: open Notepad → type a paragraph → Ctrl+S → save as test.txt → close.

■ **Outcome** Full keyboard + mouse control

Day 5

Screen Capture & Basic OCR

- Learn `mss` for fast screenshots (10x faster than `pyautogui`)
- Install `easyocr` — `pip install easyocr`
- Build `screen_agent.py`: `screenshot()`, `screenshot_region(x,y,w,h)`, `read_screen_text()`
- Test OCR accuracy on different apps — Notepad, Chrome, File Explorer

■ ■ **Build** Take screenshot of any open window, run OCR, extract all visible text into a string.

■ **Outcome** Agent can 'see' and read text from any screen

■ **Tip** *EasyOCR first load is slow (downloads model). Cache it as a global variable.*

Day 6

Voice Input & Output

- Build `tts_agent.py` using `pyttsx3` (offline) — `speak(text)`, `set_voice()`, `set_speed()`
- Build `stt_agent.py` using `SpeechRecognition` + Google STT — `listen()`, `transcribe()`
- Add hotkey to start/stop listening (e.g. Ctrl+Space)
- Test: speak a command → transcribe → print it

■ ■ **Build** Jarvis says 'Listening...' → you say 'Open Chrome' → it prints 'open chrome'.

■ **Outcome** Voice I/O pipeline working end-to-end

■ **Tip** *Use `energy_threshold=300` in `SpeechRecognition` to avoid picking up background noise.*

Day 7

Command Parser + Central Dispatcher

- Build `command_parser.py` — map natural phrases to actions using simple keyword matching
- Build `dispatcher.py` — routes parsed command to correct agent module
- Commands to handle: 'open X', 'close X', 'search X', 'type X', 'screenshot', 'read screen'
- Add a simple REPL: `while True: cmd = input('> ') → parse → dispatch → execute`

■ ■ **Build** Type 'open whatsapp' → WhatsApp opens. Type 'close chrome' → Chrome closes. Type 'screenshot' → screenshot saved.

■ **Outcome** Text command → action pipeline working

■ Week 1 Demo: A text-command interface where you type 'open chrome', 'close notepad', 'take screenshot', 'read screen' — and the agent executes each command. Record a 2-min demo video.

Week 1.5 | Browser Control Basics | Days 8–10

Day 8 | Playwright Setup + Browser Basics

- pip install playwright && playwright install chromium
- Build browser_agent.py: open_browser(), navigate(url), click(selector), type_text(selector, text), close()
- Navigate to google.com, search 'python tutorials', click first result
- Take screenshot after each action

■■ **Build** browser_agent.navigate('https://youtube.com') → page loads → screenshot saved.

■ **Outcome** Browser opens and navigates on command

■ **Tip** Use headless=False during development so you can see what's happening.

Day 9 | Browser: Search & Extract

- Build google_search(query) → returns list of {title, url, snippet}
- Build web_fetch(url) → returns page text content
- Test: search 'weather in Mumbai' → extract weather info from results
- Add to dispatcher: 'search X' → google_search → speak top result

■■ **Build** Say 'search latest news about AI' → agent searches → reads out top 3 results.

■ **Outcome** Agent can search the web and extract information

Day 10 | File Explorer Integration + Phase 1 Capstone

- Automate File Explorer with pywinauto: open_explorer(path), navigate_to(path), select_file(name)
- Add Explorer commands to dispatcher: 'open downloads', 'go to documents', 'show desktop'
- Phase 1 Capstone: wire everything together into one cohesive run loop

■■ **Build** CAPSTONE: Start the agent → it says 'Jarvis ready'. You type commands and it: opens apps, opens explorer, searches web, reads screen. All 5 modules working together.

■ **Outcome** Phase 1 complete — working automation engine with 5 core capabilities

■ **Tip** Commit everything to GitHub tonight. Tag it v0.1-core.

Phase 2 ■ LLM Brain — Natural Language Understanding

Days 11–20

Goal: Replace keyword matching with a real LLM brain. Agent understands ANY command in natural language, plans multi-step tasks, and executes them.

Week 2 | LLM Integration + Tool Calling | Days 11–17

Day 11 | Groq API Setup (Free + Fast)

- Sign up at console.groq.com — get free API key
- Install: `pip install groq`
- Build `llm_agent.py`: `ask(prompt)` → returns string response
- Test: `ask("What is the capital of India?")` — verify response
- Also set up Ollama locally as backup (`ollama pull llama3.2`)

■■ **Build** `ask("Summarize this text: [some text]")` works correctly.

■ **Outcome** Free LLM API calling working

■ **Tip** *Groq gives you llama3-70b for free at 30 req/min — more than enough for this project. No OpenAI cost needed.*

Day 12 | Tool Calling Architecture

- Define your tools as JSON schemas: `{name, description, parameters}`
- Build tool registry — every agent function registered with its schema
- Implement `tool_call_loop(user_request)` → LLM picks tool → execute → return result
- Tools to register: `open_app`, `close_app`, `web_search`, `take_screenshot`, `read_screen`, `open_file`, `list_files`

■■ **Build** `tool_call_loop("open chrome and search for python")` → LLM returns `{tool: 'open_app', args: {name: 'chrome'}}` → chrome opens.

■ **Outcome** LLM selects and calls the right tool for any command

■ **Tip** *Write very clear tool descriptions — the LLM reads them to decide which to call. Bad descriptions = wrong tool calls.*

Day 13

Intent Classification

- Build `intent_classifier(text)` — LLM returns one of: `APP_CONTROL`, `FILE_TASK`, `WEB_SEARCH`, `SYSTEM_INFO`, `CHAT`, `UNKNOWN`
 - Route each intent to the right agent module
 - Add confidence scoring — if confidence < 0.7, ask for clarification
 - Test with 20 different commands and verify correct routing
- **Build** 'Open my downloads folder' → `FILE_TASK` → `file_agent.open_explorer(downloads)`. 'What's the weather?' → `WEB_SEARCH` → `browser_agent`.

■ **Outcome** Agent correctly classifies and routes any natural language command

Day 14

Multi-Step Task Planning

- Build `planner(goal)` → returns list of steps `[[action, tool, args]]`
 - Build `executor(steps)` → runs each step in order, handles failure
 - Test with: 'Open Chrome, go to YouTube, search lo-fi music, click first video'
 - Add step-by-step status reporting — agent speaks what it's doing
- **Build** `planner('Find the weather in Delhi and open a Chrome tab with the forecast')` → 2 step plan → executes both steps.

■ **Outcome** Agent can execute multi-step workflows from a single command

■ **Tip** *Limit plan to max 8 steps for reliability. Longer plans drift and fail.*

Day 15

Conversation Memory

- Build `memory.py`: `conversation_history` list, `add_message()`, `get_context(n=10)`
 - Pass last 10 messages as context to every LLM call
 - Test: 'Open chrome' → 'now go to github' — agent remembers chrome is open
 - Add session persistence — save/load conversation from JSON file
- **Build** Multi-turn conversation: 'Open notepad' → 'Type Hello World in it' → 'Save the file as test.txt' — each step builds on the previous.

■ **Outcome** Agent remembers context across a full session

Day 16

Voice Command Integration

- Wire voice pipeline to LLM: speak → transcribe → LLM parse → tool call → speak result
- Add wake word detection: listen for 'Hey Jarvis' before processing
- Add command timeout — if no response in 5s, say 'Please try again'
- Build cancel command: say 'stop' → cancels current action

■ ■ **Build** Full voice loop: 'Hey Jarvis, open WhatsApp' → WhatsApp opens → agent says 'WhatsApp is open'.

■ **Outcome** Complete voice-to-action pipeline

■ **Tip** *Use threading for the voice listener so it doesn't block the main thread.*

Day 17

Error Recovery & Retry

- Add try/except to every tool call — return structured error {success: false, error: str}
- Build retry(fn, max_attempts=3) decorator with exponential backoff
- On failure, LLM is given the error and asked to suggest an alternative
- Test: ask for a window that doesn't exist — agent should recover gracefully

■ ■ **Build** 'Open my bank app' → app not found → agent says 'I couldn't find that app. Would you like me to search for it online instead?'

■ **Outcome** Agent fails gracefully and suggests alternatives

- Week 2 Demo: Full natural language control. Record demo: speak 5 different commands including a multi-step one. Agent executes all correctly. Post to GitHub.

Week 2.5 | WhatsApp & Messaging | Days 18–20

Day 18

WhatsApp Desktop Automation

- Open WhatsApp Desktop with subprocess
- Use pywinauto to find the search box, type contact name, press Enter
- Use pyautogui to click on the message box and type a message
- Build whatsapp_agent.py: send_message(contact, message), open_chat(contact)

■ ■ **Build** whatsapp_agent.send_message('Maa', 'Hello from Jarvis!') — message gets sent.

■ **Outcome** WhatsApp automation working

■ **Tip** *WhatsApp Desktop UI is more automatable than WhatsApp Web. Use the desktop app.*

Day 19

WhatsApp Web (Playwright) + Telegram

- Automate WhatsApp Web: log in once, save auth state, reuse session
- Build Playwright-based send: navigate, find chat, type, send
- Add Telegram Desktop support via pywinauto (same pattern as WhatsApp)
- Add to dispatcher: 'message X on WhatsApp: Y' and 'message X on Telegram: Y'

■■ **Build** Voice: 'Send a message to Rahul on WhatsApp saying I'll be late' → message sent.

■ **Outcome** Multi-platform messaging automation

Day 20

Phase 2 Capstone — Smart Assistant MVP

- Full integration test: 20 voice/text commands across all modules
- Fix any integration bugs found
- Add a simple status bar — show current task in system tray
- Record a 3-minute demo video of all capabilities

■■ **Build** CAPSTONE DEMO: 'Hey Jarvis, search for the top Python tutorials, open the first one in Chrome, take a screenshot, and send it to my friend on WhatsApp.' — Complete this 4-step task automatically.

■ **Outcome** Phase 2 complete — LLM-powered assistant with voice, messaging, and multi-step tasks

■ **Tip** *Push v0.2-llm tag to GitHub. Write a detailed README with demo GIF.*

Phase 3 ■■ System Control & Advanced Automation

Days 21–30

Goal: Control system settings, read screen intelligently, automate complex workflows in any app. Make the agent genuinely useful for daily tasks.

Week 3 | System + App Deep Control | Days 21–27

Day 21 System Control — Volume, Brightness, Wi-Fi

- Control volume: pycaw library — `get_volume()`, `set_volume(level)`, `mute()`, `unmute()`
- Control brightness: `screen_brightness_control` — `get_brightness()`, `set_brightness(level)`
- Wi-Fi control: subprocess + netsh commands — `list_networks()`, `connect(ssid)`, `disconnect()`
- Battery info, CPU%, RAM% via `psutil`

■■ **Build** Voice: 'Set volume to 50%' → volume changes. 'Turn brightness to 30%' → brightness changes.

■ **Outcome** Full system settings control

■ **Tip** *pycaw requires admin on some systems. Test first, add admin elevation if needed.*

Day 22 Clipboard & Notification System

- Build `clipboard_agent.py`: `get_clipboard()`, `set_clipboard(text)`, `get_clipboard_history()`
- Build `notifier.py` using Windows Toast notifications — `notify(title, message, icon)`
- Add notifications for every completed task and every error
- Build clipboard watcher — detect when user copies something, offer to process it

■■ **Build** Agent completes a task → Windows toast notification pops up confirming completion.

■ **Outcome** Native Windows notifications + clipboard integration

Day 23 Intelligent Screen Reading

- Upgrade screen reading: screenshot → EasyOCR → structure text by region
- Build `find_on_screen(text)` → returns coordinates of text on screen
- Build `click_on_text(text)` → finds text on screen, clicks it
- Test: 'Click the Submit button' → OCR finds 'Submit', clicks it

■■ **Build** 'Click on the OK button' → agent takes screenshot → finds 'OK' via OCR → clicks it. Works in any app.

■ **Outcome** Agent can find and click any text visible on screen

■ **Tip** *This is your fallback for any UI you can't automate with pywinauto. If you can see it, you can click it.*

Day 24

VS Code Automation

- Open VS Code, open a project folder via Command Palette
- Use pywinauto to: open terminal, type commands, open files
- Build `vscode_agent.py`: `open_project(path)`, `open_file(name)`, `run_command(cmd)`, `open_terminal()`
- Test: 'Open my project and run the tests' → VS Code opens, tests run

■ ■ **Build** Voice: 'Open my jarvis project in VS Code and run the test suite' → VS Code opens, runs pytest, agent reads output.

■ **Outcome** VS Code integration for developer workflows

Day 25

Chrome Deep Automation

- Tab management: `new_tab()`, `close_tab()`, `switch_tab(n)`, `list_tabs()`
- Bookmark current page, go back/forward, zoom in/out
- Form filling: `fill_form({field_name: value})` → finds fields by label, fills them
- Download files: trigger download, wait for completion, return file path

■ ■ **Build** Voice: 'Open three tabs — YouTube, Gmail, and GitHub, then switch to Gmail' → 3 tabs open, Gmail focused.

■ **Outcome** Full Chrome tab + navigation control

■ **Tip** *Use CDP through Playwright for the most reliable tab control — avoids UI flakiness.*

Day 26

Task Scheduler & Reminders

- Build `scheduler.py` using `APScheduler` — `schedule_task(fn, time)`, `cancel_task(id)`, `list_tasks()`
- Add reminder support: 'Remind me in 30 minutes to take a break' → toast at right time
- Daily task: auto-generate morning briefing at 9am — weather + top news + calendar
- Save scheduled tasks to JSON — survive restarts

■ ■ **Build** 'Remind me to check email in 1 hour' → 60 minutes later, toast notification appears.

■ **Outcome** Agent can schedule future tasks and reminders

Day 27

PDF & Document Handling

- Install: pdfplumber, python-docx, openpyxl
 - Build doc_agent.py: read_pdf(path), read_docx(path), read_excel(path)
 - LLM integration: 'Summarize this PDF' → read_pdf → send to LLM → speak summary
 - Test with: 'Read the resume in my Downloads and tell me the candidate's name'
- **Build** Voice: 'Summarize the PDF on my desktop' → agent reads it, LLM summarizes, agent speaks the summary.

■ **Outcome** Agent reads and understands documents

■ **Tip** *pdfplumber handles scanned PDFs poorly — add a note to use OCR mode for scanned documents.*

■ Week 3 Demo: Chain 5 different system capabilities in one voice command sequence. Record it. This is your core resume demo clip.

Week 3.5 | UI + Cleanup | Days 28–30

Day 28

Desktop Tray App (System Tray Icon)

- Install: pystray, Pillow
 - Build tray app: icon in system tray with right-click menu
 - Menu: Start Listening, Stop Listening, Open Dashboard, Settings, Exit
 - Show current agent status in tray tooltip
- **Build** Jarvis runs in background, tray icon visible. Right-click → 'Start Listening' → agent starts listening for voice commands.

■ **Outcome** Production-style background app with tray integration

■ **Tip** *This makes it look like a real product, not a script. Interviewers notice.*

Day 29

Simple Web Dashboard (React)

- Build FastAPI backend: /api/command (POST), /api/history (GET), /api/status (GET)
- Build React frontend: command input box, conversation history, agent status
- Real-time updates via WebSocket or polling every 2s
- Show live activity log: what the agent is currently doing

■ **Build** Open localhost:3000 → type a command in the web UI → agent executes on Windows → result shown in chat.

■ **Outcome** Web UI to control the agent from browser

■ **Tip** *Use shadcn/ui components for a polished look with minimal effort.*

Day 30

Phase 3 Capstone + Resume Polish

- End-to-end test of all 15+ capabilities
- Fix any remaining bugs
- Write README.md with: what it does, tech stack, features, demo GIF, setup guide
- Record 3-minute product demo video and upload to YouTube
- Write your resume bullet points for this project (see below)

■■ **Build** CAPSTONE: Single voice command that touches 4+ modules: 'Jarvis, check my downloads folder for any new PDFs, summarize the latest one, open Chrome, and email the summary to myself.'

■ **Outcome** Resume-ready project with polished README, demo video, and working app

Phase 4 ■ Production Polish & Advanced Features

Days 31–45

Goal: Add the features that turn this from a demo into a real product: persistent memory, permission system, advanced browser agent, and packaging for distribution.

Week 4 | Memory + Safety + Advanced Browser | Days 31–37

Day 31 | Persistent Long-Term Memory

- Install: chromadb (local vector DB, no API key needed)
 - Build memory.py: store_memory(text, metadata), search_memory(query, n=5)
 - Store: every completed task, user preferences, frequently opened apps, schedule
 - Test: 'What did I do yesterday?' → searches memory → LLM summarizes
- **Build** Agent remembers that you always open Chrome first thing in the morning and proactively suggests it.

■ **Outcome** Agent learns from your usage patterns

■ **Tip** *ChromaDB is fully local — no API keys, no cost, runs entirely on your machine.*

Day 32 | User Preferences & Personalization

- Build prefs.json: {name, wake_word, voice_speed, default_browser, shortcuts: {...}}
 - LLM uses preferences: 'the user's name is X, preferred browser is Y'
 - Command: 'Remember that I prefer dark mode' → stored in preferences
 - Morning greeting: uses name + time-aware greeting
- **Build** 'Remember I prefer Spotify for music' → stored. Later: 'Play music' → agent opens Spotify (not YouTube).

■ **Outcome** Personalized agent that learns user preferences

Day 33

Permission & Safety System

- Tag every tool as SAFE / NEEDS_CONFIRM / DANGEROUS
- SAFE: take screenshot, search web, read files, open apps
- NEEDS_CONFIRM: send messages, delete files, modify system settings
- DANGEROUS: delete registry, format drives (blocked entirely)
- For NEEDS_CONFIRM: show toast 'Are you sure? Press Y to confirm'

■ ■ **Build** 'Delete all files in Downloads' → agent asks 'This will delete X files. Are you sure? (Y/N)' → waits for confirmation.

■ **Outcome** Safe agent that never does destructive things without approval

■ **Tip** *This one feature separates a toy from a serious product on your resume.*

Day 34

Advanced Browser — Form Filling & Login

- Build login(site, username, password) → handles login forms automatically
- Store credentials in local encrypted file (keyring library)
- Build form_filler(form_data_dict) → fills any form by field label
- Test: auto-fill a job application form on a test site

■ ■ **Build** Voice: 'Log me in to GitHub' → Playwright opens GitHub, fills credentials from keyring, logs in.

■ **Outcome** Agent can log into websites and fill forms automatically

■ **Tip** *Use Python keyring library for secure credential storage — never store passwords in plaintext.*

Day 35

Email Automation (Gmail via Browser)

- Open Gmail in Playwright, log in, navigate inbox
- Build email_agent.py: read_emails(n=10), compose_email(to, subject, body), reply_email(id, body)
- LLM integration: 'Read my emails' → agent reads → LLM summarizes → agent speaks summary
- Add: 'Send email to X about Y' → LLM drafts email → shows for approval → sends

■ ■ **Build** Voice: 'Read my last 5 emails and summarize them' → agent reads Gmail → LLM produces summary → agent speaks it.

■ **Outcome** Gmail read/write automation

Day 36

App-Specific Workflows

- Spotify: `play_song(name)`, `pause()`, `next()`, `set_volume()`
- Calculator: `calculate(expression)` using `pyautogui/subprocess`
- File Explorer: `navigate_to(path)`, `create_folder(name)`, `sort_by(column)`
- Add 5 more apps to your registry with custom handlers

■ **Build** Voice: 'Play Believer by Imagine Dragons on Spotify' → Spotify opens and plays the song.

■ **Outcome** Rich app-specific automation library

■ **Tip** *Each app-specific handler you add makes the demo more impressive. Focus on apps you use daily.*

Day 37

Hotkey System & Quick Commands

- Global hotkeys using `pynput`: Win+J = start listening, Win+S = stop, Win+H = show history
- Quick command palette: Win+Space → text box appears → type command → execute
- Build shortcut system: 'When I say morning routine, do X, Y, Z'
- Add custom macros: user defines their own shortcuts

■ **Build** Press Win+J → 'Jarvis listening'. Press Win+Space → palette appears → type 'search python docs' → browser opens.

■ **Outcome** Keyboard shortcut system for power users

■ **Week 4 Demo:** Show the permission system in action, persistent memory recall, Gmail reading, and Spotify control. This is your 'impressive feature' reel.

Week 5 | Packaging, Distribution & Portfolio | Days 38–45

Day 38

Logging & Telemetry

- Replace all `print()` with `loguru` structured logging
- Log every command, tool call, success/failure, execution time
- Build log viewer in the web dashboard — show last 100 actions
- Add performance metrics: average response time, success rate, most used tools

■ **Build** Web dashboard shows: 47 commands today, 94% success rate, average 1.2s response time.

■ **Outcome** Production-grade logging and monitoring

Day 39

Configuration System

- Build config.yaml with all settings: LLM model, voice settings, app registry, shortcuts
- Settings UI in web dashboard: change LLM model, wake word, voice speed without code
- Environment profiles: work mode (VS Code, Chrome, Slack) vs study mode (Chrome, Spotify, Notion)
- Import/export config as JSON for backup

■ ■ **Build** Change LLM model from web UI → agent immediately uses new model without restart.

■ **Outcome** Fully configurable without touching code

■ **Tip** *This makes the project feel like a real product.*

Day 40

Packaging — PyInstaller

- Install: pip install pyinstaller
- Build Jarvis.exe — single executable, no Python installation needed
- Test on a clean Windows machine (or VM) — verify it runs
- Create installer using NSIS or Inno Setup — professional install wizard

■ ■ **Build** Download Jarvis.exe on any Windows PC → double-click → it runs.

■ **Outcome** Distributable Windows application

■ **Tip** *This is the feature that makes interviewers say 'wow'. Ship a real .exe.*

Day 41

Polish & Bug Fixes

- Full end-to-end test: run 30 different commands and log success rate
- Fix all failures found
- Improve response speed — identify slowest tool calls, optimise
- Add startup animation — ASCII art Jarvis logo in terminal + voice greeting

■ ■ **Build** Agent completes 28/30 test commands successfully (>93% success rate).

■ **Outcome** Stable, reliable agent ready for demo

Day 42

GitHub & Open Source Setup

- Write README.md: project overview, features list, tech stack, demo GIF, setup guide, architecture diagram
- Write CONTRIBUTING.md and issue templates
- Add MIT license
- GitHub Actions: auto-lint on PR, auto-build .exe on release

■ ■ **Build** GitHub repo is publicly accessible with professional README, demo video embedded.

■ **Outcome** Open-source ready project

■ **Tip** *A great README is worth more than 50 lines of code on your resume. Spend real time on it.*

Day 43

Demo Video Production

- Record 5-minute demo with OBS Studio in 1080p
- Script: introduce the project (30s), show 8 key features (4 min), show architecture (30s)
- Add captions and chapter markers in YouTube
- Create a GIF of the best moment for the README

■■ **Build** Upload to YouTube with title: 'Jarvis OS — Windows AI Agent (React + Python + LLM)'.

■ **Outcome** Portfolio-quality demo video

■ **Tip** *Demo order: voice command → app opens → web search → multi-step task → messaging → memory recall. Saves the best for last.*

Day 44

Resume & Portfolio Integration

- Write 4 resume bullet points (see template below)
- Add project to LinkedIn with demo video link
- Deploy web dashboard to Vercel/Railway (free tier)
- Write a dev.to / Medium post: 'How I built a Windows AI Agent in 45 days'

■■ **Build** Project live on GitHub, video on YouTube, dashboard deployed, blog post published.

■ **Outcome** Full portfolio presence for this project

Day 45

Day 45 — Demo Day ■

- Give the agent 10 complex real-world tasks to complete
- Record success/fail for each — aim for >85%
- Share on LinkedIn, Twitter, Reddit r/programming
- Plan Phase 5 — what to build next (see roadmap extension below)

■■ **Build** FINAL DEMO: 10 tasks spanning all modules. Target: complete 9/10 autonomously, ask for help on 1.

■ **Outcome** A working Windows AI Agent that you built yourself in 45 days. Resume-ready. Portfolio-ready. Ship it.

■ **Tip** *Day 45 is not the end. It is the version you share. Keep building.*

■ **Final Demo:** Record and publish the complete 5-minute product demo. Share on LinkedIn with the post: 'I built a Windows AI Agent from scratch in 45 days using Python, React, and LLMs. Here's how.'

Complete Tech Stack

Core	Python 3.12 · pyautogui · pywinauto · pynput · subprocess
LLM	Groq API (LLaMA3) · Ollama (local) · Tool Calling · Prompt Engineering
Browser	Playwright · BeautifulSoup · CDP
Voice	SpeechRecognition · pyttsx3 · OpenAI Whisper (optional)
Memory	ChromaDB · JSON persistence
Vision	EasyOCR · mss · Pillow
UI	React · FastAPI · WebSockets · shadcn/ui · pystray
System	pycaw · screen_brightness_control · psutil · watchdog
Docs	pdfplumber · python-docx · openpyxl
Deploy	PyInstaller · GitHub Actions · Vercel

Resume Bullet Points (copy-paste ready)

Add these to your resume under Projects. Customise the numbers based on what you actually achieve.

- Built Jarvis OS, an autonomous Windows AI Agent in Python + React that controls apps, browser, and file system via natural language voice commands using Groq LLaMA3 and tool calling
- Implemented multi-step task planning with LLM-driven intent classification, routing commands across 6 specialized agent modules (browser, file, system, messaging, document, voice)
- Built persistent vector memory with ChromaDB, permission/safety system with 3-tier action classification, and a real-time web dashboard for agent monitoring via FastAPI + React
- Packaged as a distributable Windows .exe using PyInstaller; integrated WhatsApp, Gmail, Chrome, Spotify, and VS Code automation; achieved >90% command success rate across 30 test cases

After Day 45 — What to Build Next

Day 45 is your v1.0. Now you iterate. Each phase below adds one major capability layer from the 365-day roadmap — taken in order, they transform your MVP into a full AI OS agent.

Phase 5 — 30 days	Multi-agent architecture: split into Planner, Browser, Windows, Code, and Memory agents each running independently and coordinating via a message bus.
Phase 6 — 30 days	Computer Vision: replace OCR-only screen reading with Florence-2 + OmniParser for full visual UI understanding. Agent can click anything it sees.
Phase 7 — 30 days	Coding Agent: integrate with VS Code via LSP, read/write/refactor code, run tests, commit to Git, create PRs on GitHub automatically.
Phase 8 — 30 days	Production Deployment: add OpenTelemetry, Prometheus metrics, Docker packaging, auto-updater, plugin marketplace. This is the 365-day target.

Final Words

This project is hard to fake. When an interviewer asks 'tell me about your AI agent project', you will be able to demo it live, explain every architectural decision, and talk about real failures you fixed. That is worth more than any certification.

Day 1 through 10 will be frustrating. Things won't work. That's normal. Push through. By Day 14, you'll have a working voice-controlled app — and that feeling is what keeps you going to Day 45.

Build something real. Ship it. Then make it better. ■